
Constraint Satisfaction - II

CS5491: Artificial Intelligence
ZHICHAO LU

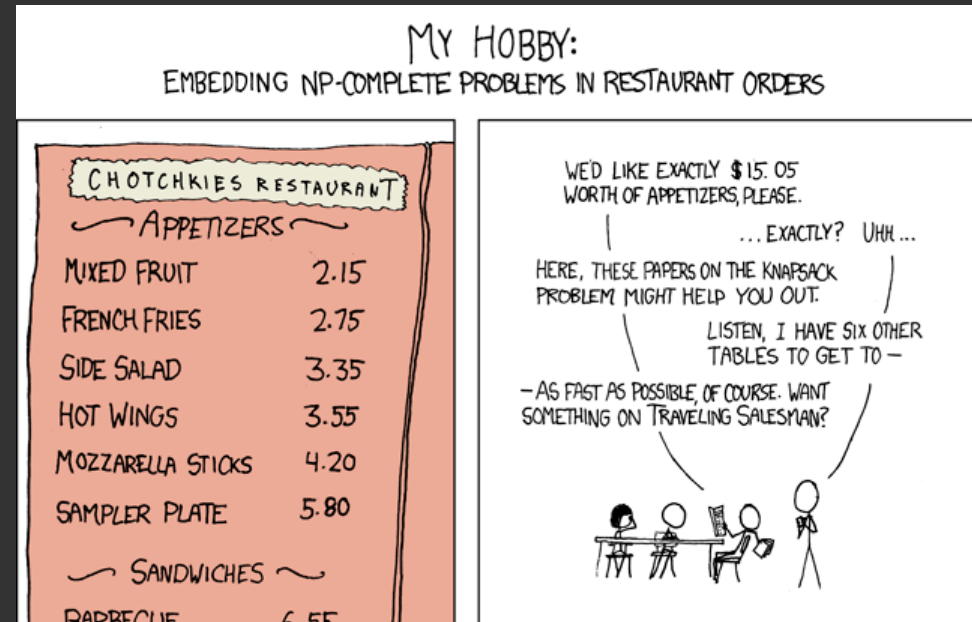
Content Credits: Prof. Wei's CS4486 Course
and Prof. Boddeti's AI Course



TODAY

Improvements to Backtracking Search:

- Filtering
- Ordering
- Problem Structure



XKCD

Reading

- Today's Lecture: RN Chapter 6

BACKTRACKING SEARCH



BACKTRACKING SEARCH



BACKTRACKING SEARCH

Backtracking search is the basic uninformed algorithm for solving CSPs



BACKTRACKING SEARCH

Backtracking search is the basic uninformed algorithm for solving CSPs

Idea 1: One variable at a time



BACKTRACKING SEARCH

Backtracking search is the basic uninformed algorithm for solving CSPs

Idea 1: One variable at a time

- Variable assignments are commutative, so fix ordering
 - Problem is commutative if order of application of any given set of actions has no effect on outcome.
- i.e., [WA = red then NT = green] same as [NT = green then WA = red]
- Only need to consider assignments to a single variable at each step



BACKTRACKING SEARCH

Backtracking search is the basic uninformed algorithm for solving CSPs

Idea 1: One variable at a time

- Variable assignments are commutative, so fix ordering
 - Problem is commutative if order of application of any given set of actions has no effect on outcome.
- i.e., [WA = red then NT = green] same as [NT = green then WA = red]
- Only need to consider assignments to a single variable at each step

Idea 2: Check constraints as you go

- i.e. consider only values which do not conflict with previous assignments
- Might have to do some computation to check the constraints
- "Incremental goal test"



BACKTRACKING SEARCH

Backtracking search is the basic uninformed algorithm for solving CSPs

Idea 1: One variable at a time

- Variable assignments are commutative, so fix ordering
 - Problem is commutative if order of application of any given set of actions has no effect on outcome.
- i.e., [WA = red then NT = green] same as [NT = green then WA = red]
- Only need to consider assignments to a single variable at each step

Idea 2: Check constraints as you go

- i.e. consider only values which do not conflict with previous assignments
- Might have to do some computation to check the constraints
- "Incremental goal test"

DFS with these two improvements is called backtracking search (not the best name)

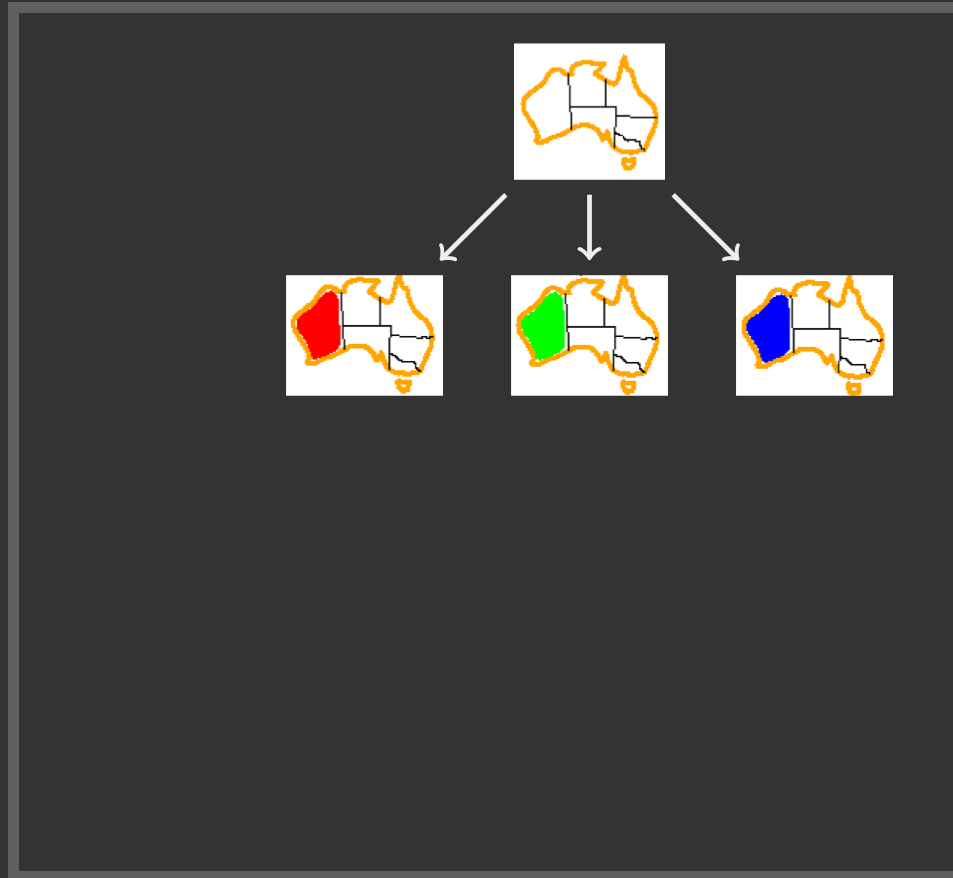
Can solve n-queens for $n \approx 25$



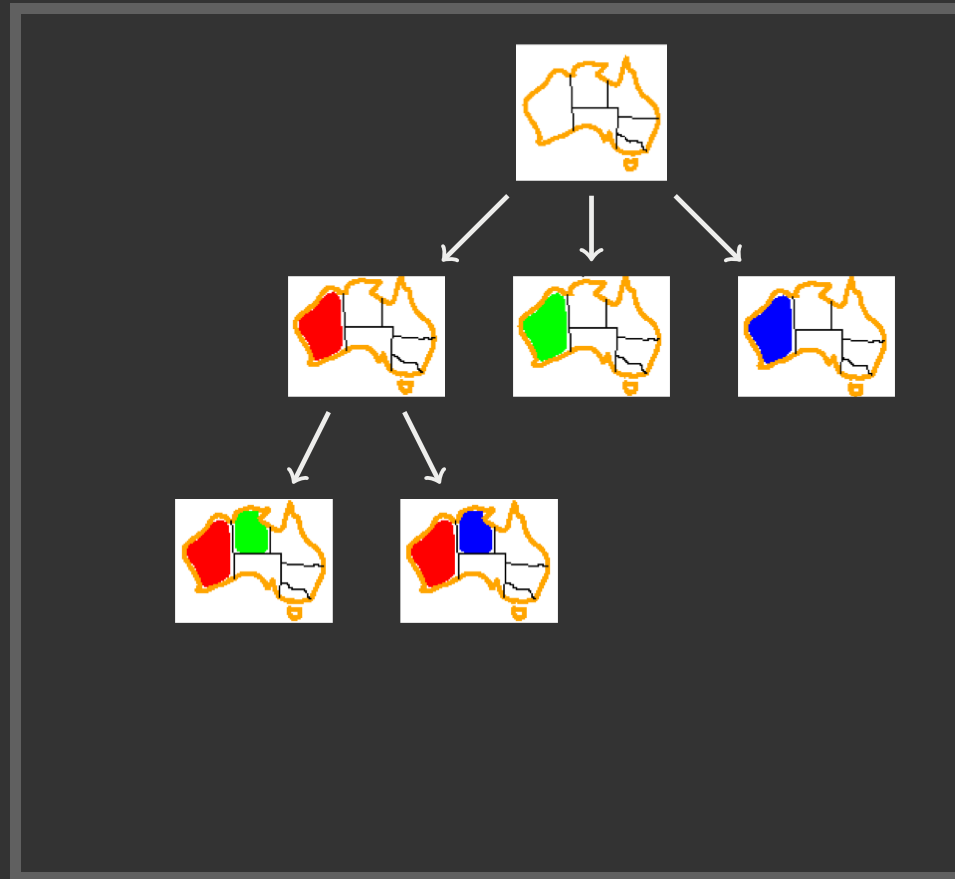
BACKTRACKING EXAMPLE



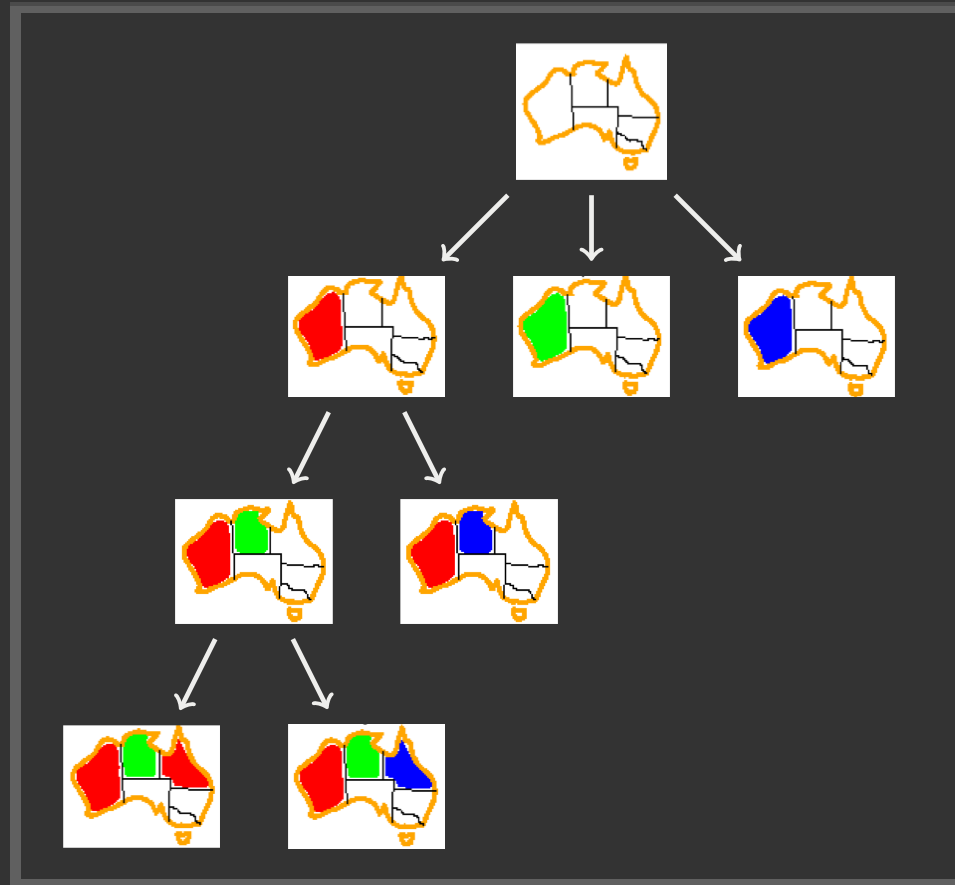
BACKTRACKING EXAMPLE



BACKTRACKING EXAMPLE



BACKTRACKING EXAMPLE



BACKTRACKING SEARCH

```
function BACKTRACKING-SEARCH(csp) returns solution/failure
  return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns soln/failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment given CONSTRAINTS[csp] then
      add {var = value} to assignment
      result ← RECURSIVE-BACKTRACKING(assignment, csp)
      if result ≠ failure then return result
      remove {var = value} from assignment
  return failure
```

Backtracking = DFS + variable-ordering + fail-on-violation



IMPROVING BACKTRACKING SEARCH



IMPROVING BACKTRACKING SEARCH

General-purpose ideas give huge gains in speed

- Backtracking is an uninformed algorithm. So we don't expect it to be very effective for large problems. We know the informed search can improve the efficiency and effectiveness.



IMPROVING BACKTRACKING SEARCH

General-purpose ideas give huge gains in speed

- Backtracking is an uninformed algorithm. So we don't expect it to be very effective for large problems. We know the informed search can improve the efficiency and effectiveness.

Ordering:



IMPROVING BACKTRACKING SEARCH

General-purpose ideas give huge gains in speed

- Backtracking is an uninformed algorithm. So we don't expect it to be very effective for large problems. We know the informed search can improve the efficiency and effectiveness.

Ordering:

- Which variable should be assigned next?
- In what order should its values be tried?



IMPROVING BACKTRACKING SEARCH

General-purpose ideas give huge gains in speed

- Backtracking is an uninformed algorithm. So we don't expect it to be very effective for large problems. We know the informed search can improve the efficiency and effectiveness.

Ordering:

- Which variable should be assigned next?
- In what order should its values be tried?

Filtering: Can we detect inevitable failure early?



IMPROVING BACKTRACKING SEARCH

General-purpose ideas give huge gains in speed

- Backtracking is an uninformed algorithm. So we don't expect it to be very effective for large problems. We know the informed search can improve the efficiency and effectiveness.

Ordering:

- Which variable should be assigned next?
- In what order should its values be tried?

Filtering: Can we detect inevitable failure early?

Structure: Can we exploit the problem structure?



ORDERING



ORDERING: MINIMUM REMAINING VALUES



ORDERING: MINIMUM REMAINING VALUES

Variable Ordering: Minimum remaining values (MRV):

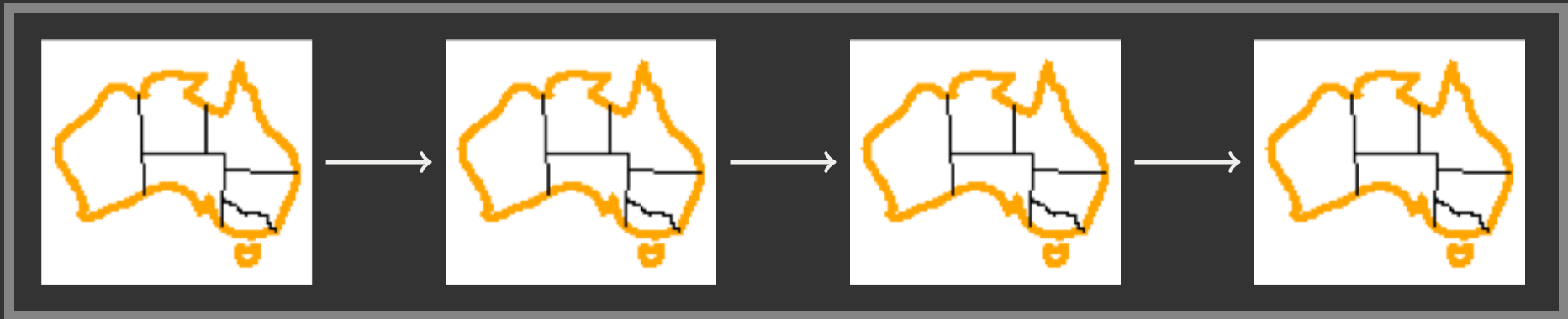
- Choose the variable with the fewest legal values left in its domain



ORDERING: MINIMUM REMAINING VALUES

Variable Ordering: Minimum remaining values (MRV):

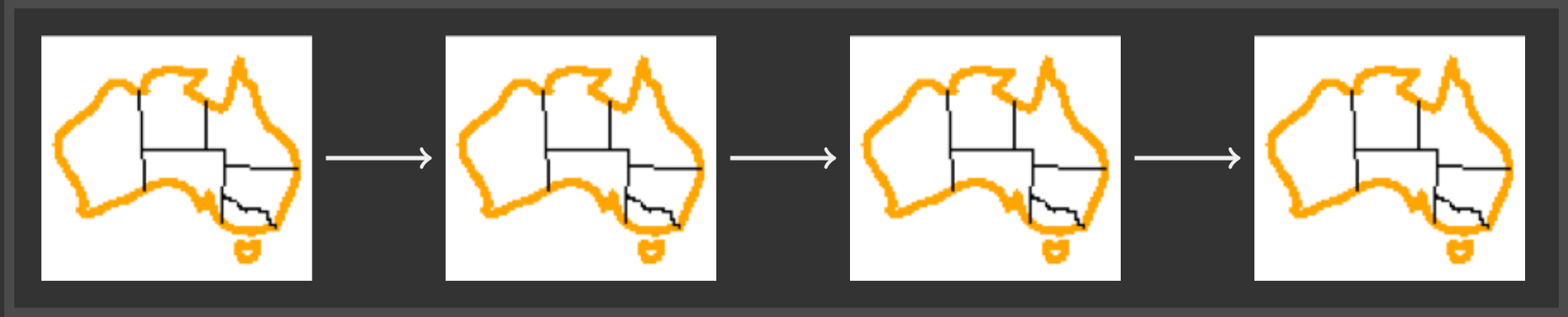
- Choose the variable with the fewest legal values left in its domain



ORDERING: MINIMUM REMAINING VALUES

Variable Ordering: Minimum remaining values (MRV):

- Choose the variable with the fewest legal values left in its domain



Why min rather than max?

ORDERING: MINIMUM REMAINING VALUES

Variable Ordering: Minimum remaining values (MRV):

- Choose the variable with the fewest legal values left in its domain



Why min rather than max?

Also called "most constrained variable"

ORDERING: MINIMUM REMAINING VALUES

Variable Ordering: Minimum remaining values (MRV):

- Choose the variable with the fewest legal values left in its domain

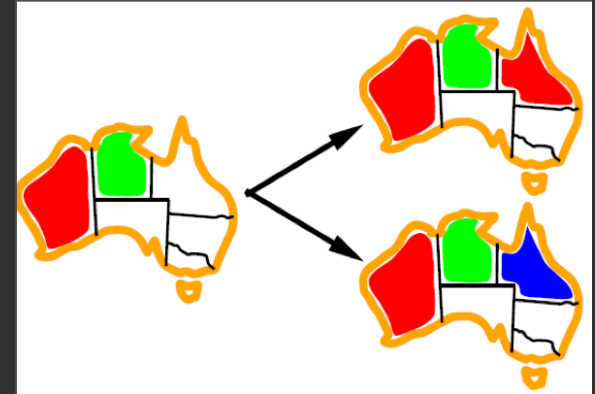


Why min rather than max?

Also called "most constrained variable"

"Fail-fast" ordering

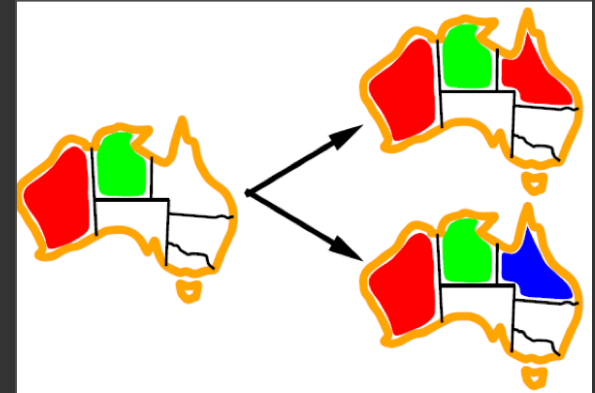
ORDERING: LEAST CONSTRAINING VALUE



ORDERING: LEAST CONSTRAINING VALUE

Value Ordering: Least Constraining Value

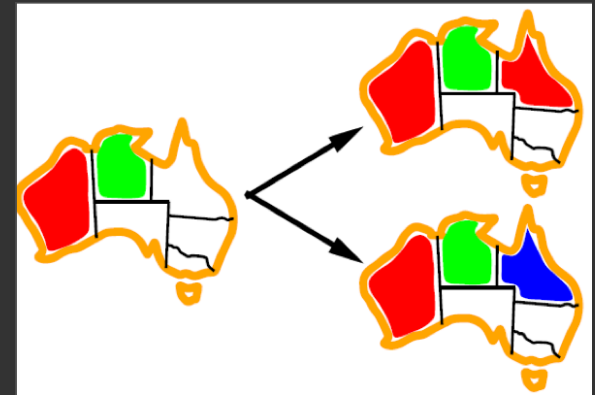
- Given a choice of variable, choose the least constraining value
- i.e., the one that rules out the fewest values in the remaining variables



ORDERING: LEAST CONSTRAINING VALUE

Value Ordering: Least Constraining Value

- Given a choice of variable, choose the least constraining value
- i.e., the one that rules out the fewest values in the remaining variables

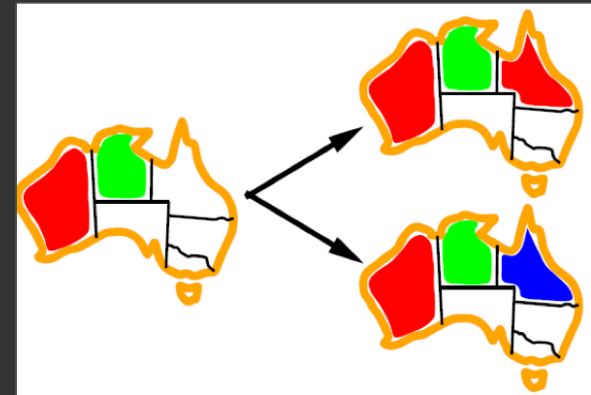


Why least rather than most?

ORDERING: LEAST CONSTRAINING VALUE

Value Ordering: Least Constraining Value

- Given a choice of variable, choose the least constraining value
- i.e., the one that rules out the fewest values in the remaining variables



Why least rather than most?

Combining these ordering ideas makes 1000 queens feasible

FILTERING



FILTERING: FORWARD CHECKING



FILTERING: FORWARD CHECKING

Filtering: Keep track of domains for unassigned variables and cross off bad options.

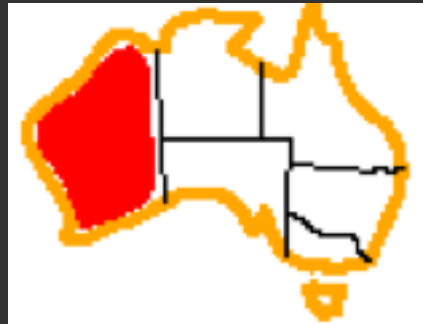


WA	NT	Q	NSW	V	SA	T

FILTERING: FORWARD CHECKING

Filtering: Keep track of domains for unassigned variables and cross off bad options.

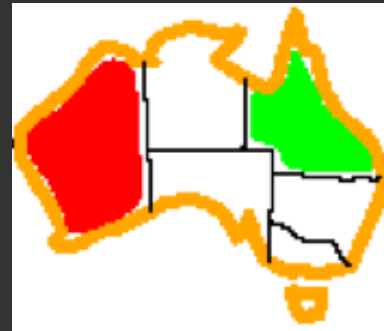
Forward checking: Cross off values that violate a constraint when added to the existing assignment



FILTERING: FORWARD CHECKING

Filtering: Keep track of domains for unassigned variables and cross off bad options.

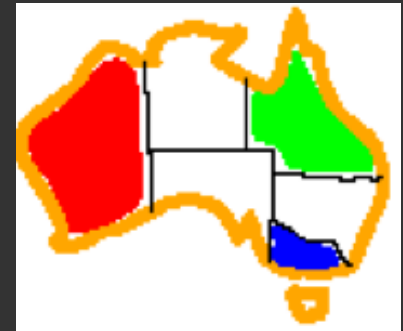
Forward checking: Cross off values that violate a constraint when added to the existing assignment



FILTERING: FORWARD CHECKING

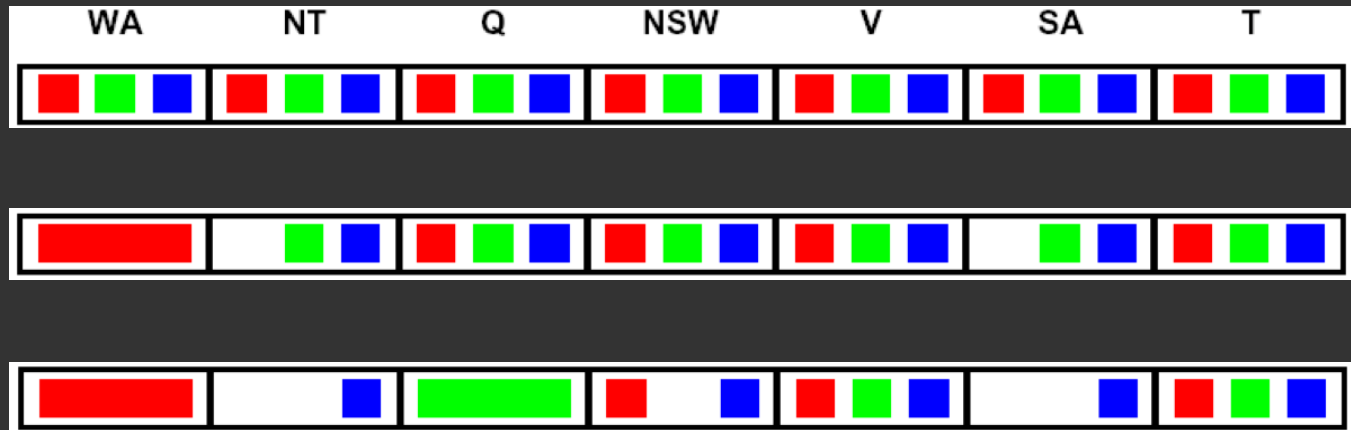
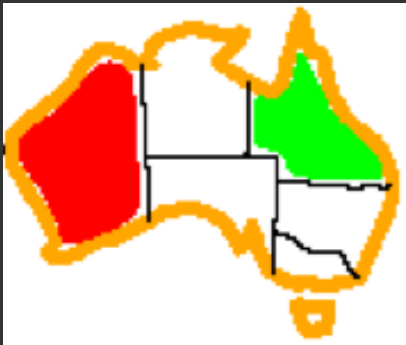
Filtering: Keep track of domains for unassigned variables and cross off bad options.

Forward checking: Cross off values that violate a constraint when added to the existing assignment



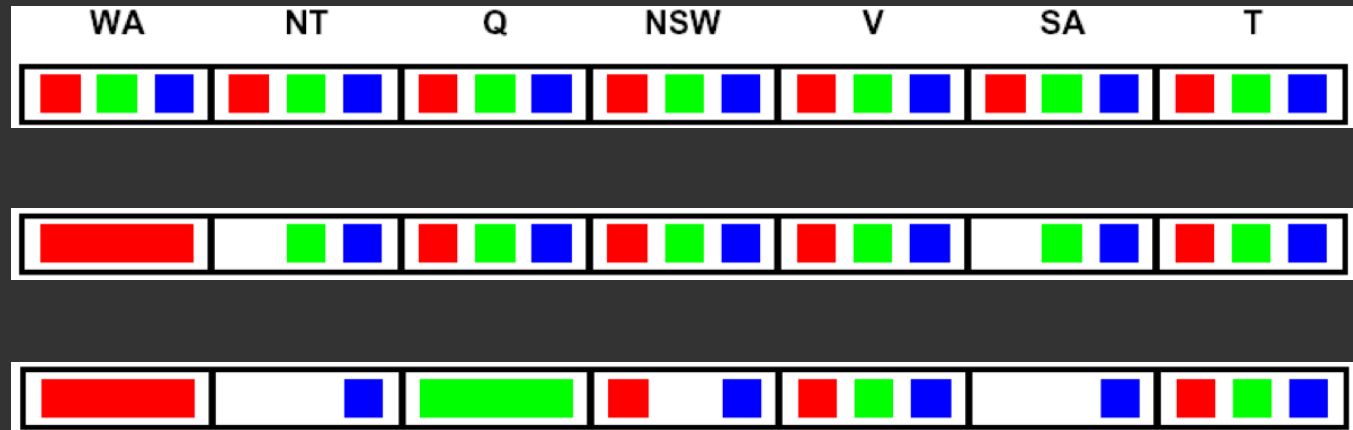
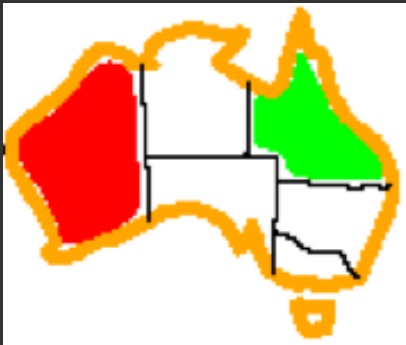
FILTERING: CONSTRAINT PROPAGATION

Forward checking propagates information from assigned to unassigned variables, but doesn't provide early detection for all failures:



FILTERING: CONSTRAINT PROPAGATION

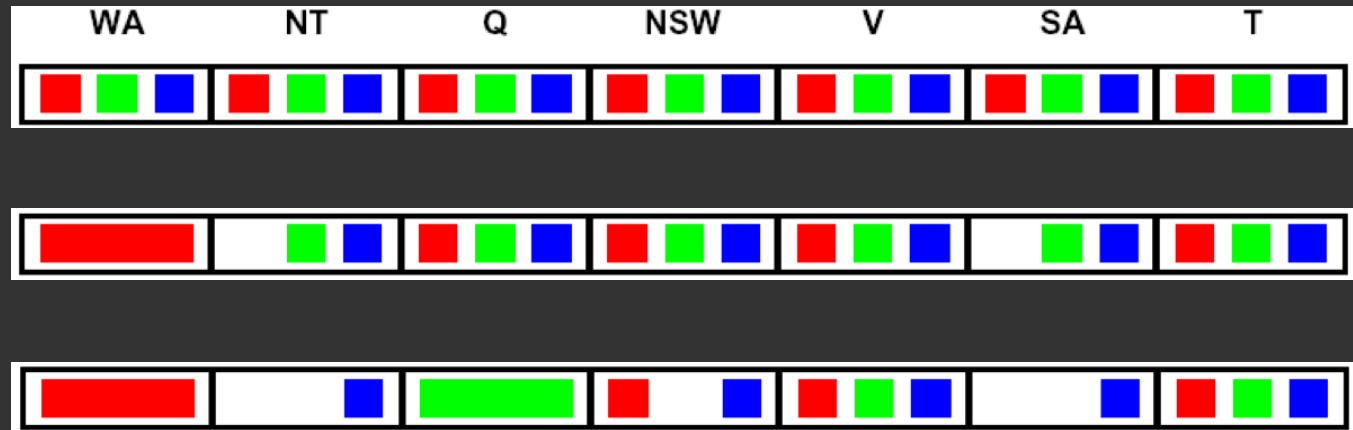
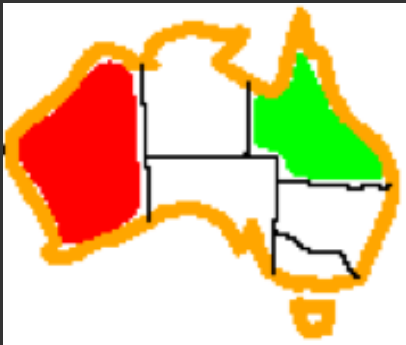
Forward checking propagates information from assigned to unassigned variables, but doesn't provide early detection for all failures:



NT and SA cannot both be blue!

FILTERING: CONSTRAINT PROPAGATION

Forward checking propagates information from assigned to unassigned variables, but doesn't provide early detection for all failures:

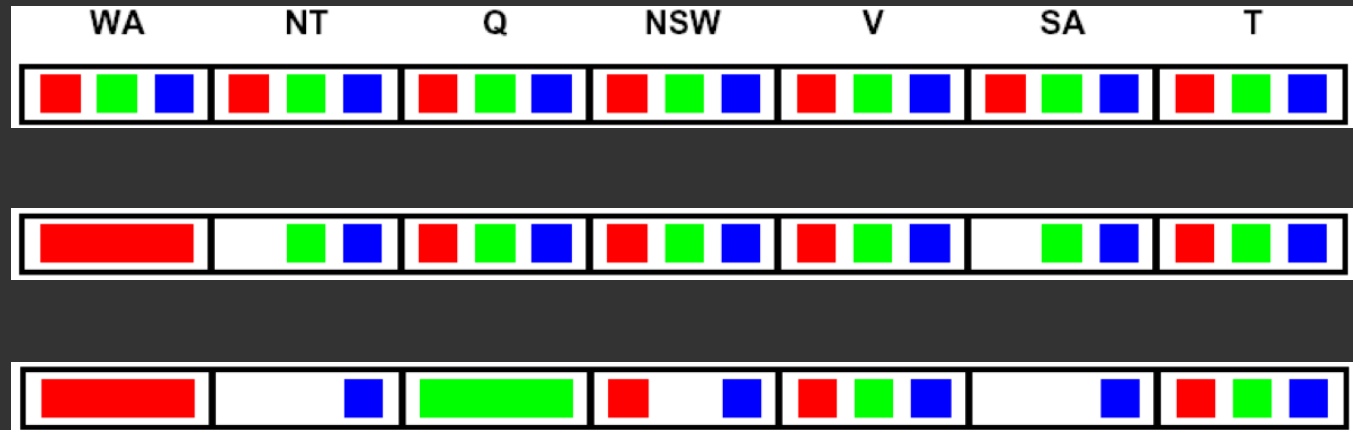
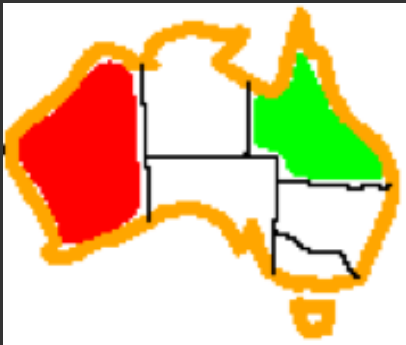


NT and SA cannot both be blue!

Why didn't we detect this yet?

FILTERING: CONSTRAINT PROPAGATION

Forward checking propagates information from assigned to unassigned variables, but doesn't provide early detection for all failures:



NT and SA cannot both be blue!

Why didn't we detect this yet?

Constraint propagation: reason from constraint to constraint

CONSISTENCY OF A SINGLE ARC

An arc $X \rightarrow Y$ is **consistent** iff for *every* x in the tail there is some y in the head which could be assigned without violating a constraint.



WA	NT	Q	NSW	V	SA
					

WA	NT	Q	NSW	V	SA
					

WA	NT	Q	NSW	V	SA
					

CONSISTENCY OF A SINGLE ARC

An arc $X \rightarrow Y$ is **consistent** iff for *every* x in the tail there is some y in the head which could be assigned without violating a constraint.



WA	NT	Q	NSW	V	SA

WA	NT	Q	NSW	V	SA

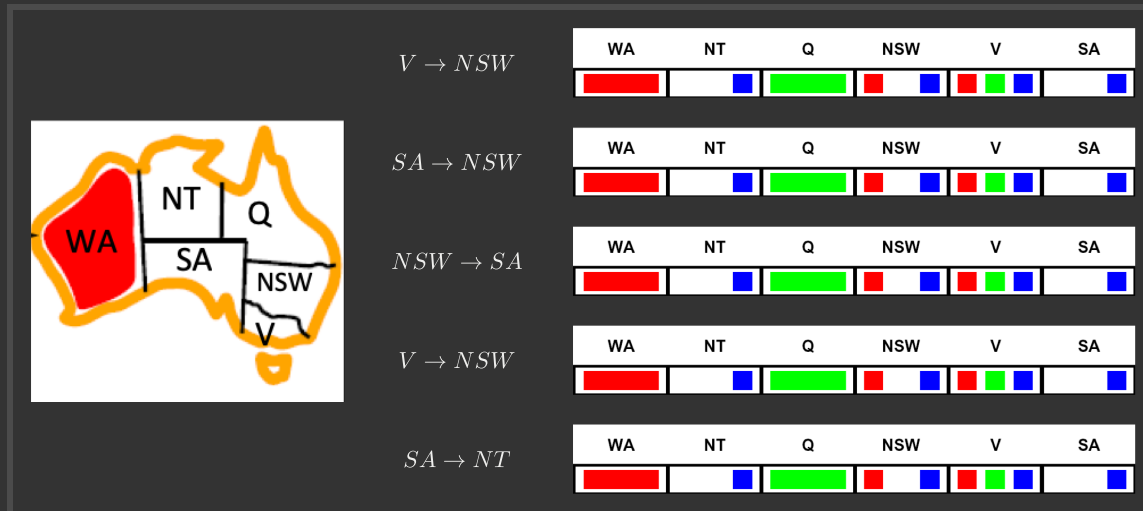
WA	NT	Q	NSW	V	SA

Forward checking: Enforcing consistency of arcs pointing to each new assignment.

 Delete from the tail !!

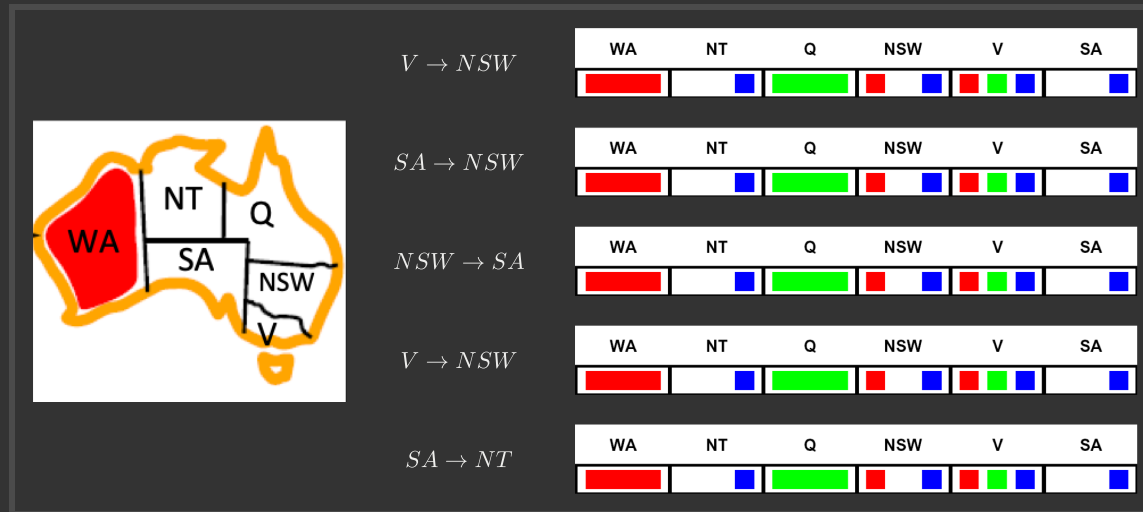
ARC CONSISTENCY OF AN ENTIRE CSP

A simple form of propagation makes sure all arcs are consistent:



ARC CONSISTENCY OF AN ENTIRE CSP

A simple form of propagation makes sure all arcs are consistent:



Important: If X loses a value, neighbors of X need to be rechecked.

Arc consistency detects failure earlier than forward checking

Can be run as a preprocessor or after each assignment

What is the downside of enforcing arc consistency?

ENFORCING ARC CONSISTENCY IN A CSP

```
function AC-3(csp) returns the CSP, possibly with reduced domains
inputs: csp, a binary CSP with variables  $\{X_1, X_2, \dots, X_n\}$ 
local variables: queue, a queue of arcs, initially all the arcs in csp

while queue is not empty do
     $(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\textit{queue})$ 
    if REMOVE-INCONSISTENT-VALUES( $X_i, X_j$ ) then
        for each  $X_k$  in NEIGHBORS[ $X_i$ ] do
            add  $(X_k, X_i)$  to queue



---


function REMOVE-INCONSISTENT-VALUES( $X_i, X_j$ ) returns true iff succeeds
    removed  $\leftarrow$  false
    for each  $x$  in DOMAIN[ $X_i$ ] do
        if no value  $y$  in DOMAIN[ $X_j$ ] allows  $(x, y)$  to satisfy the constraint  $X_i \leftrightarrow X_j$ 
            then delete  $x$  from DOMAIN[ $X_i$ ]; removed  $\leftarrow$  true
    return removed
```

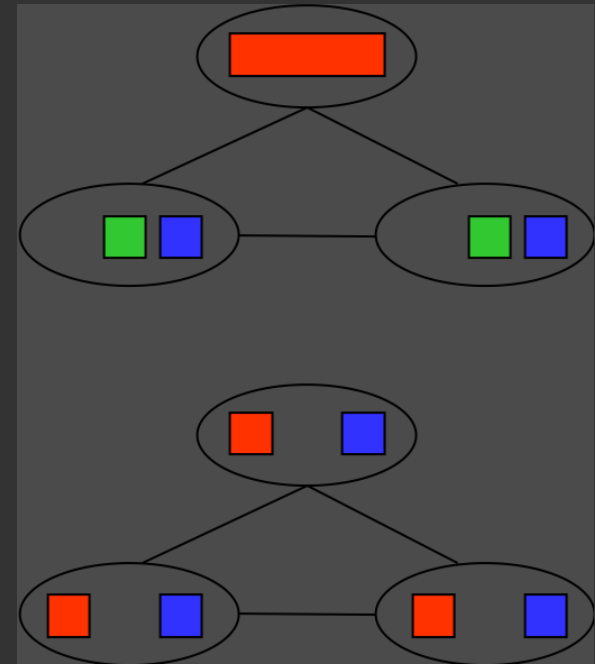
Runtime: $\mathcal{O}(n^2 d^3)$, can be reduced to $\mathcal{O}(n^2 d^2)$

But detecting all possible future problems is NP-hard.



LIMITATIONS OF ARC CONSISTENCY

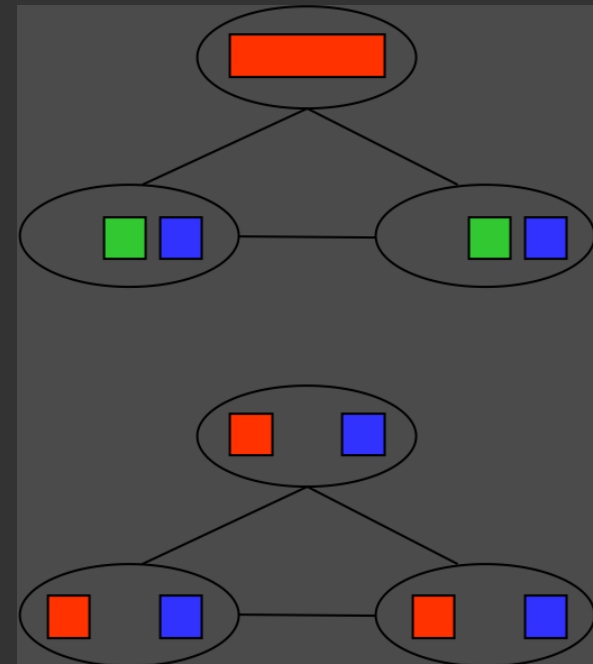
After enforcing arc consistency:



LIMITATIONS OF ARC CONSISTENCY

After enforcing arc consistency:

- Can have one solution left
- Can have multiple solutions left
- Can have no solutions left (and not know it)

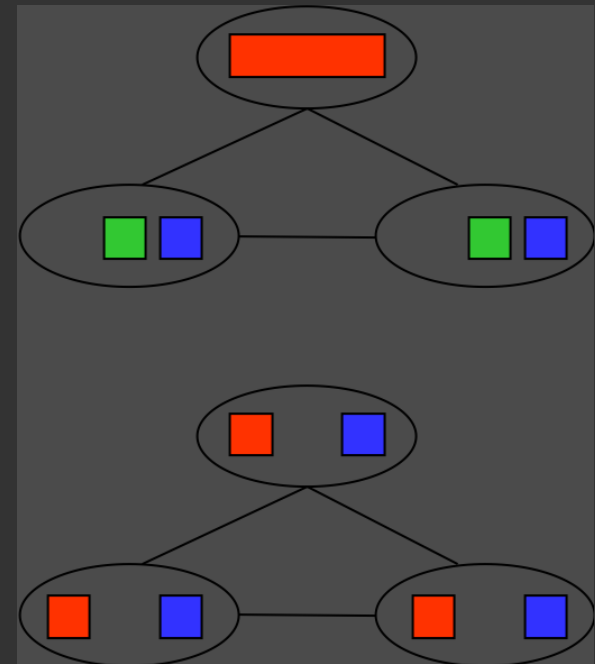


LIMITATIONS OF ARC CONSISTENCY

After enforcing arc consistency:

- Can have one solution left
- Can have multiple solutions left
- Can have no solutions left (and not know it)

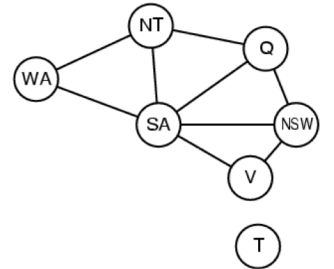
Arc consistency still runs inside a backtracking search.



PROBLEM STRUCTURE

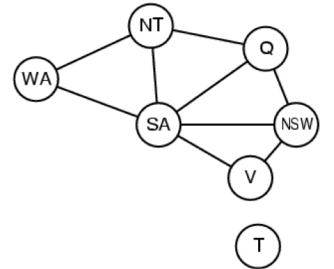


PROBLEM STRUCTURE



PROBLEM STRUCTURE

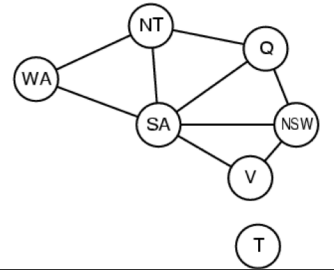
Real world problem can be decomposed into many subproblems.



PROBLEM STRUCTURE

Real world problem can be decomposed into many subproblems.

Extreme case: independent subproblems

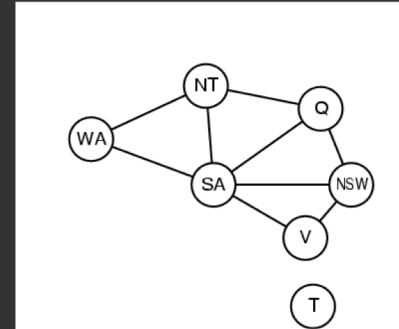


PROBLEM STRUCTURE

Real world problem can be decomposed into many subproblems.

Extreme case: independent subproblems

- Tasmania and mainland are independent subproblems



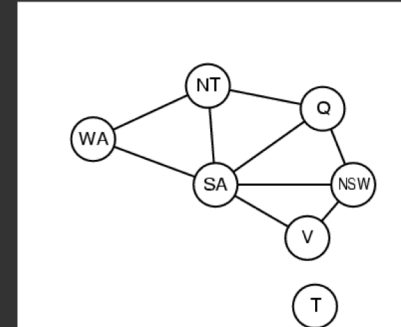
PROBLEM STRUCTURE

Real world problem can be decomposed into many subproblems.

Extreme case: independent subproblems

- Tasmania and mainland are independent subproblems

Independent subproblems are identifiable as connected components of constraint graph



PROBLEM STRUCTURE

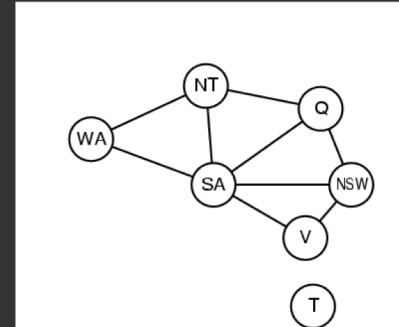
Real world problem can be decomposed into many subproblems.

Extreme case: independent subproblems

- Tasmania and mainland are independent subproblems

Independent subproblems are identifiable as connected components of constraint graph

Suppose a graph of n variables can be broken into subproblems of only c variables:



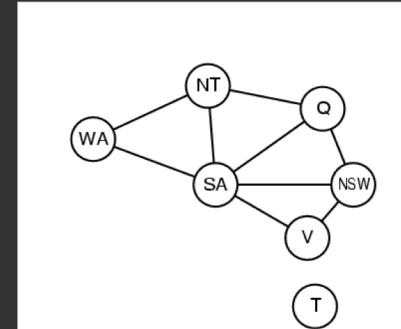
PROBLEM STRUCTURE

Real world problem can be decomposed into many subproblems.

Extreme case: independent subproblems

- Tasmania and mainland are independent subproblems

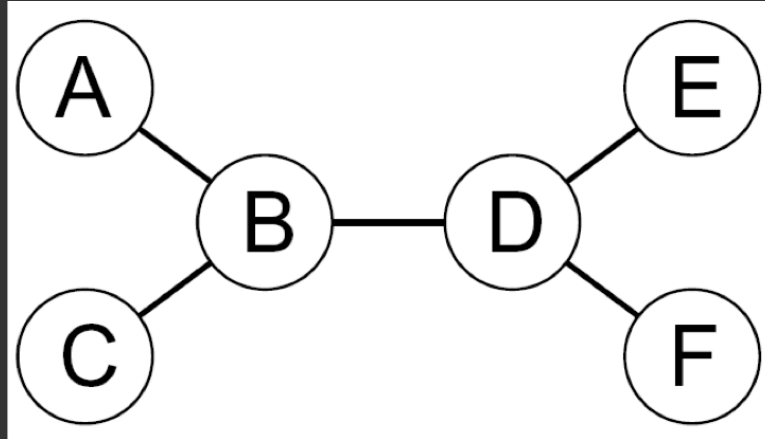
Independent subproblems are identifiable as connected components of constraint graph



Suppose a graph of n variables can be broken into subproblems of only c variables:

- Worst-case solution cost is $\mathcal{O}((n/c)(d^c))$, linear in n
- E.g., $n = 80, d = 2, c = 20$
- $2^{80} = 4$ billion years at 10 million nodes/sec
- $(4)(2^{20}) = 0.4$ seconds at 10 million nodes/sec

PROBLEM STRUCTURE



Theorem: if the constraint graph has no loops, the CSP can be solved in $\mathcal{O}(nd^2)$ time

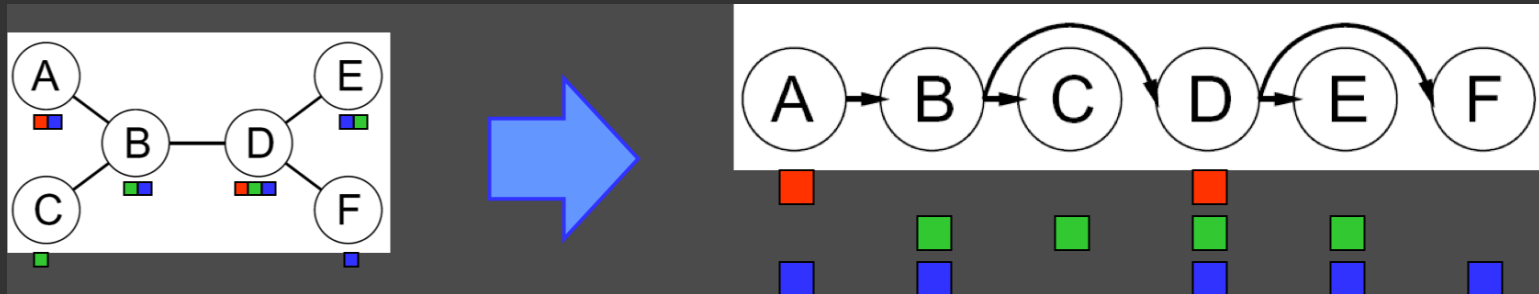
- Compare to general CSPs, where worst-case time is $\mathcal{O}(d^n)$

This property also applies to probabilistic reasoning (later): an example of the relation between syntactic restrictions and the complexity of reasoning.

PROBLEM STRUCTURE

Algorithm for tree-structured CSPs:

- Order: Choose a root variable, order variables so that parents precede children

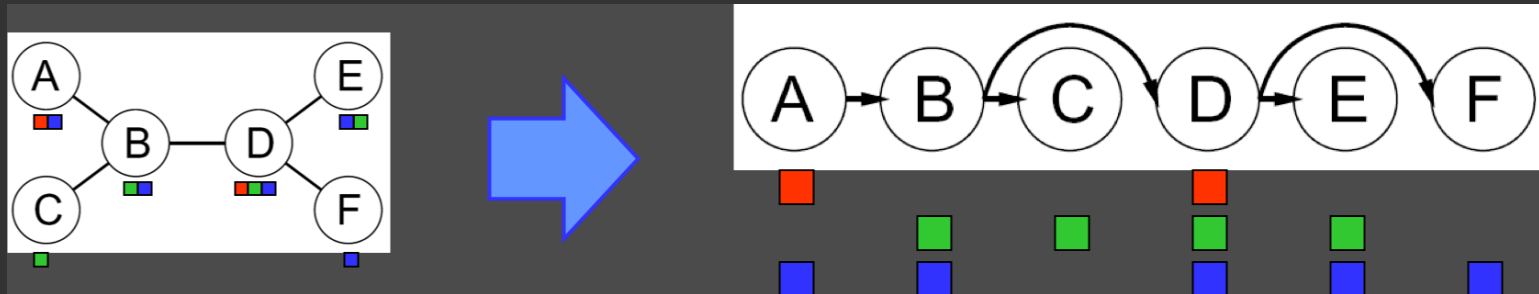


- Remove backward: For $i = n : 2$, apply $\text{RemoveInconsistent}(\text{Parent}(X_i), X_i)$
- Assign forward: For $i = 1 : n$, assign X_i consistently with $\text{Parent}(X_i)$

PROBLEM STRUCTURE

Algorithm for tree-structured CSPs:

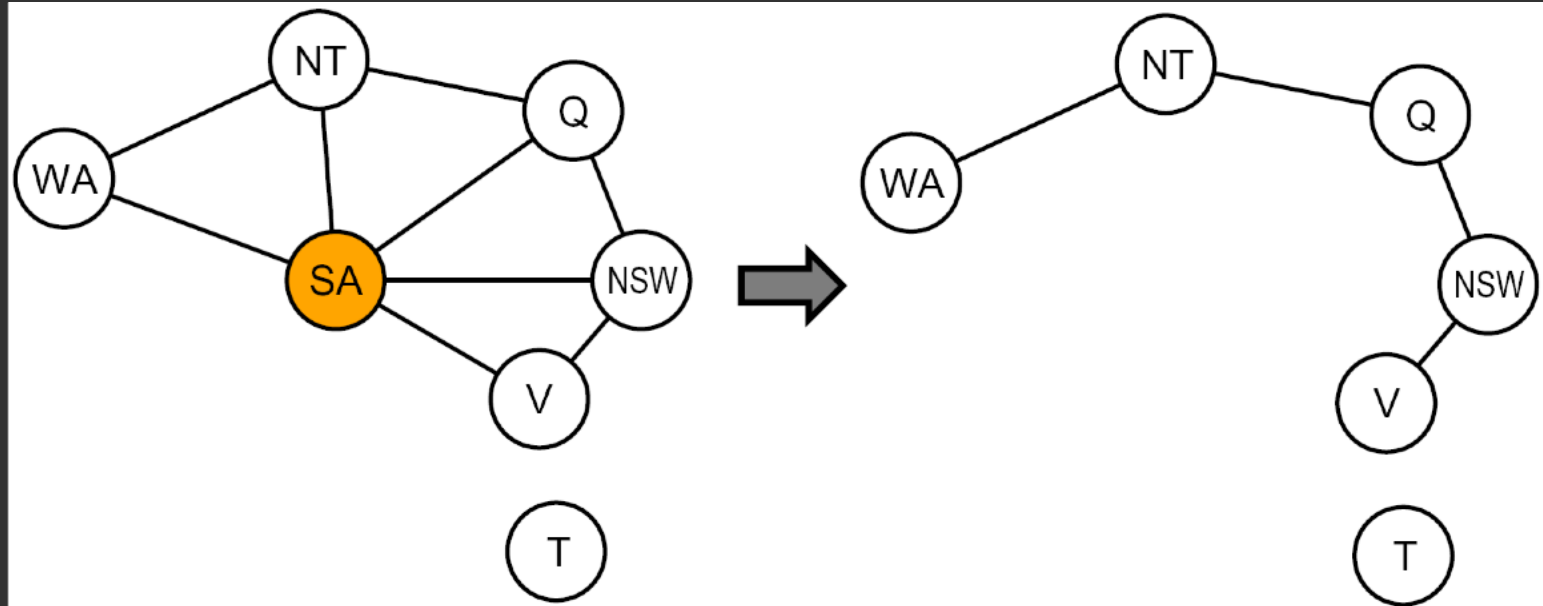
- Order: Choose a root variable, order variables so that parents precede children



- Remove backward: For $i = n : 2$, apply $\text{RemoveInconsistent}(\text{Parent}(X_i), X_i)$
- Assign forward: For $i = 1 : n$, assign X_i consistently with $\text{Parent}(X_i)$

Runtime: $\mathcal{O}(nd^2)$

PROBLEM STRUCTURE



Conditioning: instantiate a variable, prune its neighbors' domains

Cutset conditioning: instantiate (in all ways) a set of variables such that the remaining constraint graph is a tree

Cutset size c gives runtime $\mathcal{O}((d^c)(n - c)d^2)$, very fast for small c .

Q & A



XKCD

